



Getting started

SUMMARY

The ROOK Capacitor SDK allows developers to extract health data for both iOS and Android using a single codebase. To begin integration, the development environment must meet specific criteria, such as Xcode 16 for iOS and API level 26 for Android. The installation process utilizes npm and requires manual linking of pods for iOS projects. Key configuration steps include enabling HealthKit capabilities and background modes in Xcode, as well as submitting a partnership request to Samsung for production access to health data. The SDK is initialized at the top level of the application using a configuration method where developers specify the environment, client credentials, and background synchronization preferences.

What follows in the Fundamentals section of this documentation is a tour of the most important aspects of the ROOK SDK. It should cover enough for you to know how to integrate us into your application.

Minimum requirements

iOS

- `ios` ≥ 13.0
- `XCode` ≥ 26.0

Android

- `android` ≥ 26.0
- `targetSDK` ≥ 36.0
- `kotlin` $\geq 1.8.10$

Installation

```
npm install capacitor-rook-sdk  
npx cap sync
```

If you're on a Mac and developing for iOS, you need to install the pods (via CocoaPods) to complete the linking.

```
npx pod-install
```

Configure Apple Health, Health connect && Samsung Health

Apple Health (iOS)

We need to add Apple Health Kit Framework to our project in order to that please:

- Open your project in Xcode.
- Click on your project file in the Project Navigator.
- Select your target and then click on the "Build Phases" tab.
- Click on the "+" button under the "Link Binary With Libraries" section and select "HealthKit.framework" from the list.
- Select your target and then click on the "Signing Capabilities" tab.
- Click on "Add Capability" and search for "HealthKit"

Additionally add the following to the info.plist

```
<key>NSHealthShareUsageDescription</key>  
<string>This app requires access to your health and fitness data in order to track  
your workouts and activity levels.</string>  
<key>NSHealthUpdateUsageDescription</key>  
<string>This app requires permission to write healt data to HealthKit.</string>
```

Samsung Health

Android Studio Narwhal 4 Feature Drop | 2025.1.4 or higher is recommended.

Copy the [samsung-health-data.aar](#) file to your project libs directory and include it as a dependency:

```
dependencies {  
    implementation(files("$rootDir/libs/samsung-health-data-api-1.0.0-b2.aar"))  
}
```

Samsung Health Data also needs the following dependencies to work correctly:

```
dependencies {  
    implementation("com.google.code.gson:gson:2.13.0")  
}
```

Restrictions

- Samsung Health Data SDK requires Samsung Health v6.29 or later version installation.
- Samsung Health runs on devices with Android 10 (API level 29) or above. It is available on all Samsung smartphones and also non-Samsung Android smartphones.
- The SDK doesn't support an emulator.

Health Connect (Android)

We need to configure the Android project. Open the Android project inside Android Studio. We need to modify the AndroidManifest.xml file to access the Health Connect records. Add an intent filter inside your activity tag to open the Health Connect app, and your `AndroidManifest.xml` file should look like this:

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">  
  
    ...  
  
    <application  
        ...>  
        <activity  
            ...>
```

```

<intent-filter>
    ...
</intent-filter>

<!-- For supported versions through Android 13, create an activity to show
the rationale
of Health Connect permissions once users click the privacy policy link. -->
<intent-filter>
    <action
android:name="androidx.health.ACTION_SHOW_PERMISSIONS_RATIONALE" />
</intent-filter>
</activity>

<!-- For versions starting Android 14, create an activity alias to show the
rationale
of Health Connect permissions once users click the privacy policy link. -->
<activity-alias
    android:name="ViewPermissionUsageActivity"
    android:exported="true"
    android:permission="android.permission.START_VIEW_PERMISSION_USAGE"
    android:targetActivity=".MainActivity">

    <intent-filter>
        <action android:name="android.intent.action.VIEW_PERMISSION_USAGE" />
        <category android:name="android.intent.category.HEALTH_PERMISSIONS" />
    </intent-filter>
</activity-alias>
</application>
</manifest>

```

Included permissions

! INFO

This SDK will use the following permissions. There is no need to declare them in your manifest as the ROOK SDK already declares them in its own manifest.

```

<uses-permission android:name="android.permission.ACTIVITY_RECOGNITION"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE"/>
<uses-permission android:name="android.permission.FOREGROUND_SERVICE_HEALTH"/>
<uses-permission android:name="android.permission.POST_NOTIFICATIONS"/>
<uses-permission android:name="android.permission.RECEIVE_BOOT_COMPLETED"/>

```

```

<uses-permission android:name="android.permission.health.READ_SLEEP"/>
<uses-permission android:name="android.permission.health.READ_STEPS"/>
<uses-permission android:name="android.permission.health.READ_DISTANCE"/>
<uses-permission android:name="android.permission.health.READ_FLOORS_CLIMBED"/>
<uses-permission android:name="android.permission.health.READ_ELEVATION_GAINED"/>
<uses-permission android:name="android.permission.health.READ_OXYGEN_SATURATION"/>
<uses-permission android:name="android.permission.health.READ_VO2_MAX"/>
<uses-permission
  android:name="android.permission.health.READ_TOTAL_CALORIES_BURNED"/>
<uses-permission
  android:name="android.permission.health.READ_ACTIVE_CALORIES_BURNED"/>
<uses-permission android:name="android.permission.health.READ_HEART_RATE"/>
<uses-permission android:name="android.permission.health.READ_RESTING_HEART_RATE"/>
<uses-permission
  android:name="android.permission.health.READ_HEART_RATE_VARIABILITY"/>
<uses-permission android:name="android.permission.health.READ_EXERCISE"/>
<uses-permission android:name="android.permission.health.READ_SPEED"/>
<uses-permission android:name="android.permission.health.READ_WEIGHT"/>
<uses-permission android:name="android.permission.health.READ_HEIGHT"/>
<uses-permission android:name="android.permission.health.READ_BLOOD_GLUCOSE"/>
<uses-permission android:name="android.permission.health.READ_BLOOD_PRESSURE"/>
<uses-permission android:name="android.permission.health.READ_HYDRATION"/>
<uses-permission android:name="android.permission.health.READ_BODY_TEMPERATURE"/>
<uses-permission android:name="android.permission.health.READ_RESPIRATORY_RATE"/>
<uses-permission android:name="android.permission.health.READ_NUTRITION"/>
<uses-permission android:name="android.permission.health.READ_MENSTRUATION"/>
<uses-permission android:name="android.permission.health.READ_POWER"/>

```

Google may require you to provide an explanation about the

`FOREGROUND_SERVICE`/`FOREGROUND_SERVICE_HEALTH` permissions. These permissions are used by our Automatic Sync and Background Steps features to extract health data and upload it to ROOK servers. We recommend asking users for permission before enabling these features. Google may also require a video proof of a screen where a user can turn these features on or off.

Request data access

When you are developing with the Health Connect SDK, data access is unrestricted. However, to have data access when your app is launched on the Play Store, you must complete the Developer Declaration Form. More information can be found [here](#).

When you are asked about what data types your app is using, please add the following data types as **READ** access:

- ActiveCaloriesBurnedRecord
- BloodGlucoseRecord
- BloodPressureRecord
- BodyTemperatureRecord
- DistanceRecord
- ElevationGainedRecord
- ExerciseSessionRecord
- FloorsClimbedRecord
- HeartRateRecord
- HeartRateVariabilityRmssdRecord
- HeightRecord
- HydrationRecord
- MenstruationPeriodRecord
- NutritionRecord
- OxygenSaturationRecord
- PowerRecord
- RespiratoryRateRecord
- RestingHeartRateRecord
- SleepSessionRecord
- SpeedRecord
- StepsCadenceRecord
- StepsRecord
- TotalCaloriesBurnedRecord
- Vo2MaxRecord
- WeightRecord

Obfuscation

If you are using obfuscation, consider the following:

In your `proguard-rules.pro` file, add the following rule:

```
-keep class com.google.crypto.** { *; }
```

In your gradle.properties (Project level) add the following to disable R8 full mode:

```
android.enableR8.fullMode=false
```

If you want to enable full mode add the following rules to proguard-rules.pro:

```
# Keep generic signature of Call, Response (R8 full mode strips signatures from non-kept items).
-keep,allowobfuscation,allowshrinking interface retrofit2.Call
-keep,allowobfuscation,allowshrinking class retrofit2.Response

# With R8 full mode generic signatures are stripped for classes that are not kept. Suspend functions are wrapped in continuations where the type argument # is used.
-keep,allowobfuscation,allowshrinking class kotlin.coroutines.Continuation
# Crypto
-keep class com.google.crypto.** { *; }
```

Secure storage (optional)

This SDK saves sensitive information like current logged user in a password protected storage, you can specify a file name (without extension) and the password used to access it with two meta-data tags in your manifest:

! INFO

This configuration is completely optional if you don't provide the two following tags the SDK will use a default file name and auto-generate a new password every time your app is installed or it's data is deleted.

```
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
  <application>
```

```
    <!--
```

```
    PLEASE NOTE THAT BOTH TAGS ARE REQUIRED
```

```
    IF YOU EVER LOSE THE PASSWORD THE SDK WILL BECOME UNABLE TO PERFORM ITS
```

FUNCTIONS

YOU CAN CHANGE THE NAME OF THE STORAGE TO FIX IT, BUT ALL PREVIOUS DATA WILL BE LOST.

-->

```
<meta-data
    android:name="io.tryrook.security.storage.FILE_NAME"
    android:value="secure_storage"/>
```

```
<meta-data
    android:name="io.tryrook.security.storage.PASSWORD"
    android:value="my_super_secret_password"/>
```

```
</application>
```

```
</manifest>
```

Completed AndroidManifest

At the end your **AndroidManifest.xml** should look like:

```
<?xml version="1.0" encoding="utf-8"?>
<manifest xmlns:android="http://schemas.android.com/apk/res/android">
    <application
        android:name=".RookMonoApplication"
        android:icon="@mipmap/ic_launcher"
        android:label="@string/app_name"
        android:theme="@style/AppTheme">
        <activity
            android:name=".features.HomeActivity"
            android:exported="true">
            <intent-filter>
                <action android:name="android.intent.action.MAIN"/>

                <category android:name="android.intent.category.LAUNCHER"/>
            </intent-filter>
        </activity>

        <!-- For supported versions through Android 13, create an activity to show
the rationale
of Health Connect permissions once users click the privacy policy link. -->
        <activity
            android:name=".features.privacypolicy.HCPrivacyPolicyActivity"
            android:enabled="true"
            android:exported="true">
```



```

        <intent-filter>
            <action
android:name="androidx.health.ACTION_SHOW_PERMISSIONS_RATIONALE"/>
        </intent-filter>
    </activity>

    <!-- For versions starting Android 14, create an activity alias to show the
rationale
of Health Connect permissions once users click the privacy policy link.
-->
    <activity-alias
        android:name="ViewPermissionUsageActivity"
        android:exported="true"
        android:permission="android.permission.START_VIEW_PERMISSION_USAGE"

        android:targetActivity=".features.privacypolicy.HCPrivacyPolicyActivity">

        <intent-filter>
            <action android:name="android.intent.action.VIEW_PERMISSION_USAGE"/>
            <category
android:name="android.intent.category.HEALTH_PERMISSIONS"/>
        </intent-filter>
    </activity-alias>
</application>
</manifest>

```

Samsung Health Request data access

During development, you can access Samsung Health data by enabling [developer mode](#).

To have data access in the production environment, you must submit the Samsung's Partnership Request [Form](#).

Please submit partner request before your app distribution. Your app's package name and signature (SHA-256) will be registered in the Samsung Health's system after an approval.

Ensure that the correct app signature and package name is registered. If your published app signature differs from what you have provided, your app will not have access to Samsung's data.

You will see a section like this, select only the data types under the "Read" section:

Supported Read Data Types*

☐ Active calories goal
 ☐ Active time goal
 ☐ Activity summary (total active calories burned, active time, calories burned, and distance)
 ☐ Blood glucose

☐ Blood oxygen
 ☐ Blood pressure
 ☐ Body composition
 ☐ Energy score

☐ Exercise
 ☐ Exercise location
 ☐ Floors climbed
 ☐ Heart rate

☐ Nutrition
 ☐ Nutrition goal
 ☐ Skin temperature
 ☐ Sleep

☐ Sleep goal
 ☐ Steps
 ☐ Steps goal
 ☐ Water intake

☐ Water intake goal
 ☐ User profile

Please select the data types your app will use

Supported Write Data Types

☐ Blood glucose
 ☐ Blood pressure
 ☐ Body composition
 ☐ Heart rate

☐ Nutrition
 ☐ Water intake

Please select the data types your app will use

This SDK uses the following data types, however you don't need to declare all of them if you don't use it.

Configure and initialize rook sdk in your app

Add your client UUID to be authorized. Follow the example below and add the **RookConfig** at the top level of your main app. The password refers to the secret key.

```
import { RookConfig } from 'capacitor-rook-sdk';
import { useEffect, useState } from 'react';

const Home: React.FC = () => {
  useEffect(() => {
    RookConfig.initRook({
      environment: 'sandbox',
      clientUUID: 'YOUR-CLIENT-UUID',
      secret: 'YOUR-SECRET-WORD',
      bundleId: 'YOUR-CUSTOM-BUNDLE-ID',
      packageName: 'YOUR-CUSTOM-PACKAGE-NAME',
      enableBackgroundSync: true,
    })
      .then(() => console.log('Initialized'))
      .catch((e: any) => console.log('error', e));
  }, []);
  ...
}
```

The parameters `bundleId` and `packageName` are strings optional, if those parameters are not set, the sdk will take the bundle id or `packageName` of the current applicaiton.

Note: secret parameter is the value configured in your portal account